فصل اول

فصل اول: محاسبات موازی بر روی سیستم‌های ناهمگون.....۲

۱ موازی‌سازی.....۲

۲ معماری کامپیوتر.....۴

۳ محاسبات ناهمگون یا **HETEROGENEOUS**.....۷

۴ معماری **heterogeneous** یا نامتجانس (ناهمگون).....۷

۵ قابلیت محاسباتی یا **compute capabilitiy**.....۱۰

۶ الگوی محاسباتی ناهمگون.....۱۰

۷ **CUDA** : یک پلتفرم برای محاسبات ناهمگون.....۱۲

۸ آیا برنامه‌نویسی با **CUDA** مشکل است؟.....۱۷

۹ محیط توسعه‌ی **CUDA**.....۱۸

فصل دوم : مدل برنامه‌نویسی CUDA.....۲۰

۱ معرفی مدل برنامه‌نویسی **CUDA**.....۲۰

۲ ساختار برنامه‌نویسی **CUDA**.....۲۱

۳ مدیریت حافظه.....۲۳

۴ سلسله مراتب حافظه **GPU** (خلاصه).....۲۵

۱ تفاوت فضای حافظه **host** و **device**.....۲۸

۵ مدیریت تردها.....۲۸

فصل اول: محاسبات موازی بر روی سیستم‌های ناهمگون

۱ موازی‌سازی

امروزه، موازی سازی در همه‌جا وجود دارد، و برنامه‌نویسی موازی به عنوان یک مبحث جدایی ناپذیر در دنیای برنامه‌نویسان به شمار میآید. موازی‌سازی در چندین سطوح، مرتبط با معماری طراحی می باشد. به صورت کلی دو نوع موازی‌سازی وجود دارد:

- موازی‌سازی **Task**
- موازی‌سازی **Data**

موازی‌سازی **Task** وقتی کاربرد پیدا می‌کند که چندین **task** و یا چندین تابع وجود دارد که می‌توانند به صورت مستقل از همدیگر کار کنند. در واقع موازی‌سازی در مورد **Task** تلاش می‌کند که توابع را بر روی هسته‌های مختلف تقسیم کند.

موازی‌سازی **Data** وقتی کاربرد پیدا می‌کند که داده‌های زیادی وجود دارد که امکان پردازش همزمان بر روی آن‌ها وجود دارد. در واقع موازی‌سازی **Data** بر روی تقسیم داده‌ها بر روی چندین هسته تمرکز دارد.

برنامه‌نویسی **CUDA** برای مسایلی که در آن‌ها امکان محاسبات موازی بر روی داده‌ها وجود دارد، بسیار مناسب به شمار میآید. عمده تمرکز این کتاب نحوه حل برنامه نویسی موازی بر روی داده ها با استفاده از برنامه‌نویسی **CUDA** می‌باشد. بسیاری از برنامه‌هایی که برای پردازش داده‌های بزرگ هستند می‌توانند از مدل **data-parallel** برای افزایش سرعت محاسبات خود بهره ببرند. پردازش **data-parallel** داده‌ها را بر روی تردهای موازی نگاشت می‌کند.

اولین گام در طراحی برنامه **data-parallel** این است که داده‌ها را بر روی تردها پخش کنیم، که هر ترد وظیفه‌ی کار بر روی قسمتی از داده را بر عهده میگیرد. در حالت کلی، دو رویکرد برای تقسیم دیتا وجود دارد: تقسیم **block** ی و تقسیم **cyclic** . در تقسیم **block** ی، بسیاری از المان‌های متوالی از داده، به صورت دسته‌ای در کنار هم هستند. هر دسته بر روی یک ترد انتساب داده شده، و تردها در هر لحظه فقط یک دسته از این داده‌ها را پردازش می‌کنند. در تقسیم **cyclic** ،میزان داده کمتری در دسته‌ها قرار

میگیرد، پس طبیعتا تعداد دسته‌ها بیشتر از روش **block** ی خواهد بود. تردهای همسایه، دسته‌های همسایگی را نیز دریافت می‌کنند و هر ترد می‌تواند بیش از یک دسته را پردازش کند. انتخاب یک دسته جدید برای پردازش بر روی ترد، موجب پریدن هر چند دسته که در ترد است می‌شود. تصویر زیر دو مثال از نحوه‌ی تقسیم دیتای یک بعدی را نمایش میدهد. در تقسیم **block** ی، هر ترد تنها یک قسمت از داده‌ها را برای پردازش برمیدارد، و در تقسیم **cyclic** ، هر ترد بیشتر از یک دسته را برای پردازش برمیدارد.(هر رنگ مرتبط با یک ترد است، پس در این مورد چهار ترد وجود دارد).

Block partition: each thread takes one data block

Cyclic partition: each thread takes two data blocks

در تصویر زیر سه مثال ساده از دیتای دو بعدی را نشان میدهد: تقسیم **block**ی در راستای محور **y**، تقسیم **block** ای در راستای هر دو محور، و تقسیم **cyclic** در راستای محور **x** . بقیه‌ی حالت‌های ممکن به عنوان تمرین به عهده خودتان گذاشته می‌شود.

Block partition on one dimension *Block partition on both dimensions* *Cyclic partition on one dimension*

معمولا، داده به صورت یک بعدی ذخیره می‌شود، حتی اگر یک دید چند بعدی از داده داشته باشیم، همچنان بر روی حافظه به صورت یک بعدی نگاشت می‌شود. مشخص کردن این که چطور داده بر روی تردها تقسیم شود معمولا وابسته به این است که به صورت فیزیکی چطور ذخیره شده است، همچنین این که ترتیب تردها به چه صورت می‌باشد. این که چطور تردها را مدیریت می‌کند تاثیر زیادی بر روی عملکرد کلی برنامه خواهد داشت.

عملکرد برنامه معمولا حساس به **block size** هست. مشخص کردن بهترین میزان تقسیم در روش **block** ی و در روش **cyclic** به معماری کامپیوتر وابسته است. در این مورد در ادامه بیشتر یاد خواهیم گرفت.

۲ معماری کامپیوتر

چندین روش برای طبقه‌بندی کردن معماری کامپیوترها وجود دارد. یکی از این روش‌ها که خیلی استفاده می‌شود طبقه‌بند فلین است، که معماری کامپیوتر را به چهار دسته مختلف تقسیم می‌کند، این طبقه بندی بر اساس نحوه‌ی جریان یافتن **instructions** و **data** بر روی هسته‌های سیستم عامل هست که شامل موارد زیر می‌شود:

- Single Instruction Single Data (SISD)
- Single Instruction Multiple Data (SIMD)
- Multiple Instruction Single Data (MISD)
- Multiple Instruction Multiple Data (MIMD)

معماری یک دستورالعمل یک داده اشاره به کامپیوترهای قدیمی دارد: یک معماری سریال. که تنها یک هسته در چنین سیستم عاملی وجود دارد. و در هر لحظه تنها یک جریان دستورالعمل قادر به اجراست، و عملیات‌ها بر روی یک جریان داده اعمال می‌شوند.

یک دستورالعمل چند داده اشاره به نوعی از معماری موازی دارد. در این معماری چندین هسته وجود دارد. همه‌ی هسته‌ها یک جریان دستورالعمل را در هر لحظه اجرا می‌کنند، و هر کدام بر روی جریان داده‌های مختلف اجرا می‌شوند. کامپیوترهای برداری معمولا از نوع **SIMD** هستند، و بیشتر کامپیوترهای مدرن یک معماری **SIMD** را به کار میگیرند. شاید بزرگترین مزیت **SIMD** این است که، در لحظه نوشتن کد بر روی **CPU** ، برنامه‌نویس می‌تواند به صورت سریال فکر کند ولی به پرفرمنس یک عملیات موازی دست پیدا کند چرا که کامپایلر جزییات پیاده‌سازی را برعهده خواهد گرفت.

چندین دستورالعمل یک داده به یک معماری غیر رایج اشاره دارد، که هر هسته بر روی یک جریان داده‌ی یکسان با چندین جریان دستورالعملی متفاوت عملیاتی را انجام میدهد.

چندین دستورالعمل چندین داده به معماری رایج موازی سازی اشاره دارد که در آن چندین هسته بر روی چندین جریان داده عملیاتی را انجام میدهد، و هر کدام به صورت مستقل دستورالعمل‌هایی را اجرا می‌کنند. بسیاری از معماری‌های **MIMD** یک کامپوننت زیرین **SIMD** را نیز در خود دارند.

در سطح معماری، پیشرفت‌های زیادی به منظور رسیدن به موفقیت‌های زیر صورت پذیرفته است:

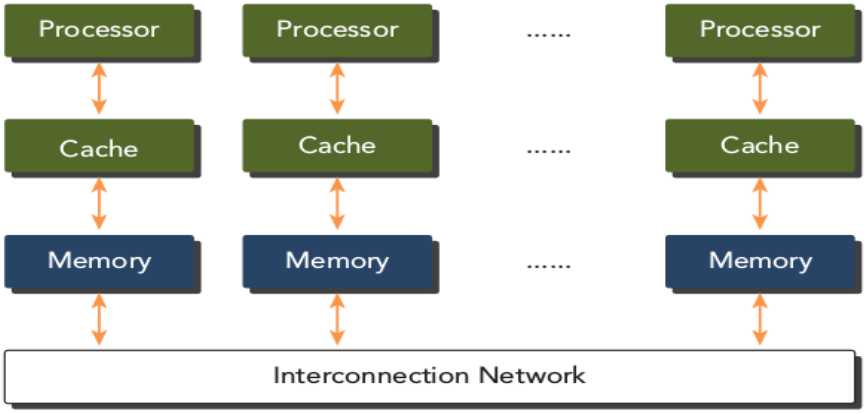
- کاهش تاخیر
- افزایش پهنای باند
- افزایش توان عملیاتی

تاخیر یا latency به میزان زمانی اطلاق می‌شود که یک عملیات شروع و پایان می‌پذیرد، و معمولا در واحد میکروثانیه بیان می‌شود. **پهنای باند** مقدار داده‌ای است که می‌تواند در یک واحد زمانی پردازش شود، معمولا پهنای باند را تحت **megabytes/sec** و یا **gigabytes/sec** بیان می‌کنند. **توان عملیاتی** میزان دستوراتی است که می‌توان در یک واحد زمانی انجام داد، که معمولا تحت عنوان **gflops** بیان می‌شود(**giga float operation per second**) ، مخصوصا در فیلدهای علمی که محاسبات سنگین بر روی محاسبات ممیز شناور انجام می‌شود. تاخیر زمان سپری شده برای تکمیل شدن یک عملیات را نشان میدهد، در حالی که توان عملیاتی تعداد دستوراتی است که می‌توان در یک واحد زمانی انجام داد را نمایش میدهد.

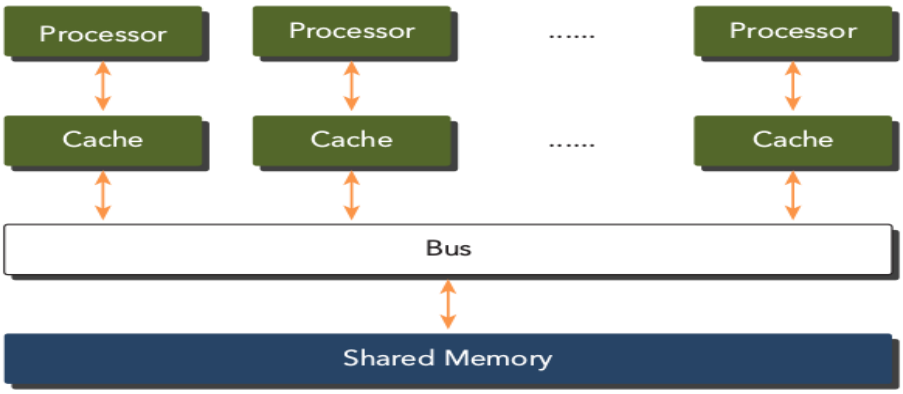
معماری کامپیوتر همچنین می‌تواند بر اساس سازماندهی حافظه تقسیم شود، بر این مبنا کامپیوترها به دو دسته تقسیم می‌شوند.

- **Multi-node with distributed memory**(چند نودی با حافظه توزیع شده)
- **Multiprocessor with shared memory**(چندپردازشگری با حافظه اشتراکی)

در یک سیستم چند نودی، موتورهای محاسباتی با ابعاد بزرگ ساخته شده که با شبکه به هم متصل هستند. هر پردازشگر حافظه محلی خود را دارد، و پردازشگرها می‌توانند محتوای حافظه محلی خود را بر روی شبکه به اشتراک بگذارند. شکل زیر یک سیستم چند نودی با حافظه توزیع شده را نمایش میدهد. چنین سیستم‌هایی را معمولا کلاستر می‌نامند.



معماری چند پردازنده‌ای (چند هسته‌ای) معمولا از رنج دو پردازنده‌ای(**dual processor**) تا هزاران یا صدها پردازنده می‌باشد. این پردازنده‌ها به صورت فیزیکی به یک حافظه مشترک متصل هستند، که در شکل بعدی مشخص می‌باشد، یا این که با یک لینک با تاخیر پایین(مثل **PCI-Express** یا **PCIe**) به هم متصل هستند.



اصطلاح **many-core** معمولا برای توصیف معماری چندهسته‌ای مخصوصا با تعداد هسته‌های بالا (ده ها یا صدها هسته) استفاده می‌شود. اخیرا، معماری کامپیوترها در حال انتقال از **multi-core** به **many-core** می‌باشد.

GPU ها یک معماری **many-core** دارند، و به صورت **مجازی** همه‌ی انواع موازی‌سازی که قبلا بیان شد را دارند: **SIMD**، **MIMD**، **multi-threading** و موازی سازی در سطح دستورات. **NVIDIA** از عبارت **single Instruction,Multiple thread(SIMT)** برای چنین معماری استفاده می‌کند.

CPU و **GPU** تاریخچه‌ی متفاوتی دارند. در واقع، **GPU** ها در گذشته به عنوان یک شتاب‌دهنده استفاده میشد. ولی اخیرا **GPU** ها خیلی قدرتمند، همه‌کاره، کاملا برنامه‌پذیر، قابلیت موازی‌سازی **task** و **data** را پیدا کرده‌اند و برای مسایلی که درگیر محاسبات موازی هستند بسیار ایده‌آل هستند.

تفاوت GPU و CPU در این دیدگاه

اگر چه ممکن است هم برای **CPU** و هم برای **GPU** از اصطلاح **many-core** استفاده شود ولی **GPU** از لحاظ معماری کاملا متفاوت از یک هسته **CPU** است.

یک هسته‌ی **CPU**، نسبتا سنگین وزن بوده، و برای محاسبات منطقی خیلی پیچیده طراحی شده است، هدف در این طراحی بهینه کردن محاسبات سریال و پشت سر هم می‌باشد.

یک هسته‌ی **GPU** ، نسبتا سبک وزن، و برای کارهای **data-parallel** با منطق ساده‌تر بهینه شده است، و تمرکز روی توان عملیاتی در برنامه‌های موازی است.

۳ محاسبات ناهمگون یا HETEROGENEOUS

در روزهای آغازین دنیای کامپیوتر، کامپیوترها تنها شامل واحد پردازنده مرکزی بودند یعنی **CPU** ها که به منظور اجرای کارهای برنامه‌نویسی کلی طراحی شده بودند. ولی در دهه‌ی گذشته، کامپیوترهای پیشرو در محاسبات با کارآیی بالا، برخی المان‌های پردازشی دیگری را نیز استفاده کردند. شاخص‌ترین آن **GPU** هست، که ذاتا برای انجام محاسبات گرافیکی به صورت موازی طراحی شده است. در طی زمان، **GPU** ها قوی‌تر و جامع‌تر شده، و باعث شد که بتوان از آن‌ها برای مقاصد کلی در کارهای محاسباتی استفاده کرد یعنی کارهای **general-purpose** که این کار با یک کارآیی فوق العاده و توان مصرفی بهینه انجام می‌شود.

معمولا، **CPU** ها و **GPU** ها به عنوان واحدهای پردازشی مجزایی هستند که با باس **PCI-Express** و تنها یک نود محاسباتی به هم متصل می‌شوند. در این معماری، **GPU** ها را به عنوان یک دیوایس مجزا می‌بینند.

تغییراتی که منجر به استفاده از سیستم‌های ناهمگون به جای سیستم‌های همگون شد راهی بود که برای رسیدن به محاسبات با کارآیی بالا در تاریخ طی شد. سیستم‌های همگون از یک یا چندین پردازنده از یک معماری یکسان برای اجرای یک برنامه استفاده میکردند. سیستم‌های ناهمگون از معماری**های** مناسب برای اجرای برنامه استفاده می‌کنند، و هر کار متناسب با معماری به پردازنده تخصیص داده می‌شود، که منجر به افزایش کارآیی در این گونه سیستم‌ها می‌شود.

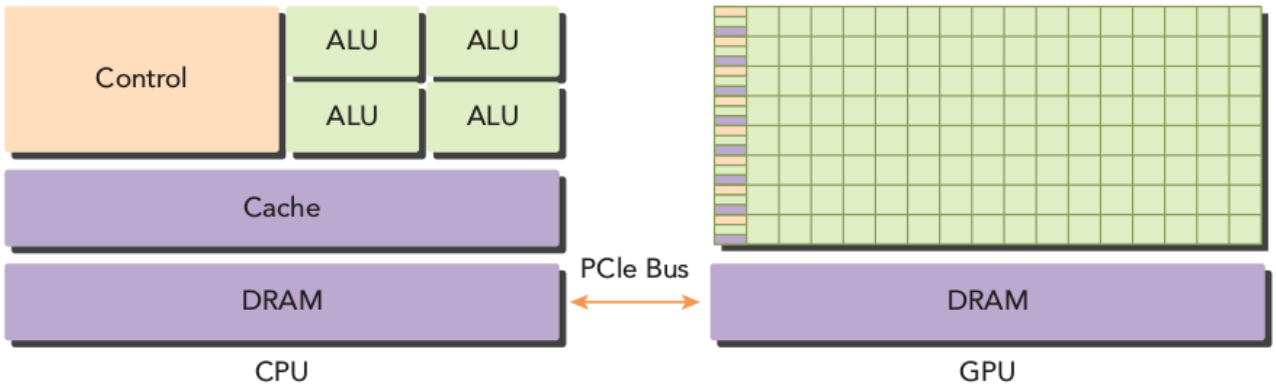
اگر چه سیستم‌های ناهمگون مزیت‌های خیلی مهمی نسبت به سیستم‌های قدیمی در محاسبات دارند، ولی امروزه استفاده موثر از این سیستم‌ها با افزایش پیچیدگی طراحی برنامه‌ها محدود شده است. در حالی که برنامه‌نویسی موازی توجه‌ها را به خود جلب کرده است، همچنان پیچیدگی‌هایی نیز در خود گنجانده است.

اگر شما در زمینه‌ی برنامه‌نویسی موازی تازه‌کار هستید، در این صورت می‌توانید از ابزارهای پیشرفته و توسعه‌هایی که در زمینه‌ی سیستم های ناهمگون رخ داده است استفاده لازم را ببرید. اگر که یک برنامه‌نویس سطح بالا در زمینه‌ی پردازش موازی هستید، تطابق یافتن با معماری ناهمگون برای شما آسان‌تر خواهد بود.

۴ معماری heterogeneous یا نامتجانس (ناهمگون)

امروزه، نودهای محاسباتی در یک معماری عادی ناهمگون متشکل از دو سوکت **CPU** و دو یا بیشتر **GPU** است. یک **GPU** دیگر یک پلتفرم مستقل نیست بلکه یک پردازنده کمکی برای **CPU** می‌باشد. بنابراین، **GPU** ها باید به عنوان یک دیوایس برای **CPU** با استفاده از یک باس **PCI-Express** باشند، که

در شکل زیر نمایش داده شده است. دقیقا به همین دلیل است که، در اصطلاح برنامه‌نویسی **GPU** مـــا از **CPU** به عنوان **host** و از **GPU** به عنوان **device** استفاده می‌کنیم.



در **heterogeneous** ها برنامه از دو بخش تشکیل میگردد.

- کد میزبان **(Host code)**
- کد دیوایس **(Device code)**

Host code بر روی **CPU** ها اجرا شده و **device code** بر روی **GPU** ها اجرا می‌شود. یک برنامـه کـه بـر روی یک پلتفرم **heterogeous** اجرا می‌شود، معمـولا بـا **CPU** شـروع می‌شـود. کـد **CPU** مسـوولیت‌های روبه‌رو را دارد: مدیریت محیط، کد، و داده‌ای است که دیوایس میگیرد.

با برنامه‌های محاسباتی سنگین، معمولا **program section** مقدار قابل توجهی از داده‌هـای مـوازی را در خود خواهد داشت. **GPU** ها برای سرعت بخشیدن به اجرای این قسمت از داده‌ی موازی هسـتند. وقـتی که یک قطعه‌ی سخت‌افزاری که به صورت سخت‌افزاری نـیز از **CPU** جـدا می‌باشـد بـه منظـور سـرعت بخشیدن به این قسمت‌ها استفاده شود، معمولا از آن به عنوان یک شتاب دهنده یاد می‌شود. **GPU** ها به عنوان بارزترین شتابدهنده‌ی سخت‌افزاری می‌باشد.

پلتفرم محاسباتی **NVIDA's GPU** در سری‌های زیر فعال هستند:

- Tegra
- GeForce
- Quadro
- Tesla

خانواده‌ی **Tegra** به منظور استفاده در موبایل‌ها و ابزارهای نهفته مانند تبلت‌ها و گوشی‌ها طراحی شده، **Gefore** برای اسـتفاده کـاربران، **Quadro** بـرای مصورسـازی‌های حـرفه‌ای و **Tesla** بـرای دیتاسـنترهای محاسبات موازی هستند. **Fermi**، شتاب‌دهنده‌ی **GPU** از خانواده‌ی **Tesla**، اخـیرا توجهـات زیـادی را بـه عنوان شتاب‌دهنده‌ی محاسباتی در برنامه‌های با کارآیی بالا جلب کـرده اسـت. **Fermi**، توسـط **NVIDIA** در سال ۲۰۱۰ به عنوان اولین معماری کامل محاسباتی **GPU** رونمایی شد. **GPU** شـتاب‌دهنده‌ی **Fermi**

تـا بـه امـروز در بسـیاری از زمینه‌هـای محاسـباتی اسـتفاده شـده اسـت مثـل مطالعـات زمین‌لـرزه، شبیه‌سازی‌های بیوشیمی، مدل‌سازی آب و هوا، و پردازش سیگنال، محاسبات مالی، آنـالیز داده. **Kepler**، نسل فعلی معماری **GPU** که بعد از **Fermi** بود، در پاییز ۲۰۱۲ رونمایی شد، که قـدرت پردازشـی خیلی بیشتری را نسبت به نسل قبلی پیشنهاد می‌کرد و روش‌های جدیدی برای بهبود و بهینه کردن اجراهـای موازی در اختیار قرار میداد. **Tegra K1** از **Kepler** به منظور افزایش قدرت محصولات نهفته استفاده کرده است.

ویژگی‌های مهمی وجود دارد که مشخص کننده‌ی توانایی **GPU** می‌باشد:

- تعداد هسته‌های **CUDA**
- میزان حافظه

همچنین، دو متریک برای اندازه‌گیری کارآیی یک **GPU** وجود دارد:

- **peak computational performance**
- **memory bandwidth**

peak computational performance راهی برای اندازه‌گیری قابلیت محاسباتی است، که به این صـورت تعریف می‌شود که چقدر محاسبات ممیز شناور **single** یا **double** را می‌توان در یک ثانیه انجام داد. **Peak performance** معمولا به صورت **gflops** و یا **tflops** بیان می‌شود. پهنای باند حافظه با **gigabytes/sec** بیان می‌شود. جدول زیر یک خلاصه‌ای از معماری **Fermi** و **Kepler** و ویژگی‌های کارآیی را آورده است.

	FERMI (TESLA C2050)	KEPLER (TESLA K10)
CUDA Cores	448	2 x 1536
Memory	6 GB	8 GB
Peak Performance*	1.03 Tflops	4.58 Tflops
Memory Bandwidth	144 GB/s	320 GB/s

* Peak single-precision floating point performance

بیشتر مثال‌هایی که در این کتاب زده می‌شود هم بر روی **Fermi** قابـل اجـرا خواهـد بـود و هم بـر روی **Kepler**. برخی از مثال‌ها نیازمند یک معماری خاص می‌باشد کـه تنهـا در **GPU** هـای **Kepler** گنجانـده شده است.

۵ قابلیت محاسباتی یا compute capabilitiy

NVIDIA از یک اصطلاح خاص یعنی **compute capability** برای ورژن سخت‌افزار **GPU** برای همه‌ی خانواده‌ی **Tesla** استفاده می‌کند. دیوایس‌ها با ورژن کلی یکسان معماری هسته یکسانی خواهند داشت.

- معماری کلاس **Kepler** با عدد ۳ نمایش داده می‌شود.
- معماری کلاس **Fermi** با عدد ۲ نمایش داده می‌شود.
- معماری کلاس **Tesla** با عدد ۱ نمایش داده می‌شود.

اولین کلاس **GPU** هایی که توسط **NVIDIA** رونمایی شد، در خود نام **Tesla** را برای همه‌ی **GPU** ها نیز داشت (یعنی هم نام **Tesla** بود و هم معماری سخت‌افزار هسته **Tesla** بود). همه‌ی مثال‌های این کتاب نیاز به **cp** بالای ۲ خواهند داشت.

TABLE 1-2: Compute Capabilities of Tesla GPU Computing Products

GPU	COMPUTE CAPABILITY
Tesla K40	3.5
Tesla K20	3.5
Tesla K10	3.0
Tesla C2070	2.0
Tesla C1060	1.3

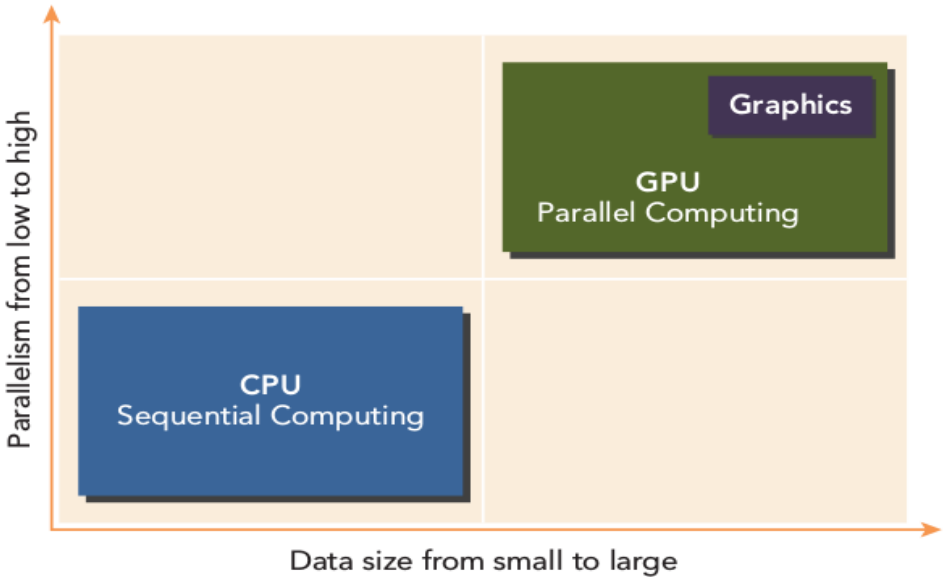
۶ ۶ الگوی محاسباتی ناهمگون

محاسبات **GPU** قصد جایگزینی با محاسبات **CPU** را ندارند. هر کدام مزیت‌های خاصی برای نوع خاصی از برنامه‌ها دارند. محاسبات **CPU** برای عملیات‌هایی که کنترل سختی دارند مناسب می‌باشد، و **GPU** برای محاسباتی مفید هستند که در آن **Task** ها محاسبات موازی زیادی دارند. وقتی **CPU** همراه با **GPU** قرار میگیرد، یک ترکیب خیلی قوی را ایجاد خواهند کرد. **CPU** برای محاسبات کوتاه که کنترل غیر قابل پیش‌بینی دارند مناسب می‌باشد، و **GPU** برای محاسبات سنگین ولی کنترل سبک مناسب می‌باشد. در واقع دو پارامتر برای انتخاب بهتر بین **CPU** و **GPU** وجود دارد:

- سطح موازی سازی
- اندازه داده

اگر یک مساله حجم داده کمی داشته باشد، و منطق برنامه پیچیده باشد، و یا سطح موازی‌سازی پایینی داشته باشد، در این صورت **CPU** انتخاب مناسب‌تری می‌باشد چرا که توانایی مناسبی در منطق پیچیده‌تر دارد. اگر یک برنامه دارای مقدار زیادی داده بود، و سطح موازی‌سازی بالایی داشته باشد، در

این صورت **GPU** انتخاب مناسب تری می‌باشد چرا که **GPU** تعداد هسته‌های بسیار بیشتری دارد، که می‌تواند از **multi-threading** به صورت گسترده استفاده کند، و نسب به **CPU** دارای عرض پهنای باند بیشتری می‌باشد.



معماری محاسبات موازی ناهمگون **CPU+GPU** به این دلیل به وجود آمد که **CPU** و **GPU** هر کدام ویژگی‌های مکملی دارند که باعث می‌شود برنامه‌ها برای هر چیزی مهیا باشند. بنابراین، به منظور کارآیی بهینه ممکن است که نیاز داشته باشید برای برنامه خودتان همه از **CPU** و هم از **GPU** استفاده کنید، اجرای قسمت ترتیبی و یا قسمت موازی بر روی **CPU** و قسمتی که دارای داده‌ی موازی سنگین هست بر روی **GPU** که در شکل زیر نمایش داده شده است.

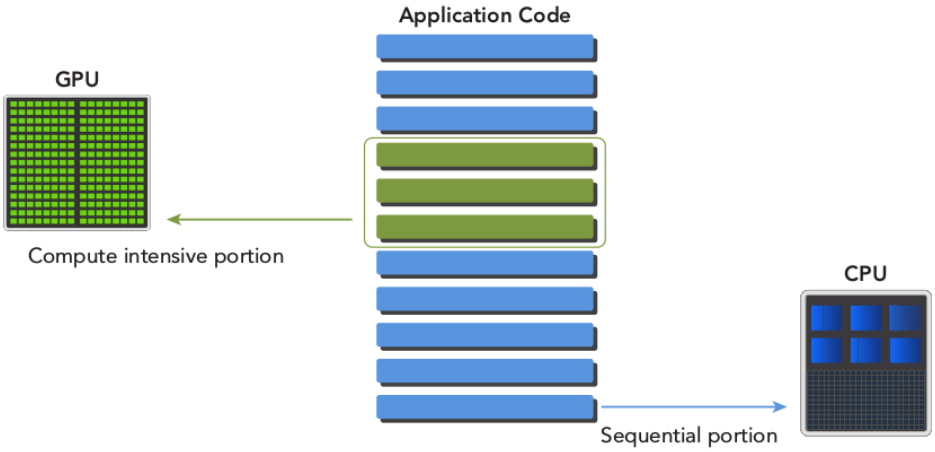


FIGURE 1-11

نوشتن کد به این صورت به شما این اطمینان را میدهد که ویژگی‌های **GPU** و **CPU** مکمل همدیگر باشند، و منجر به استفاده‌ی حداکثری از قدرت **CPU+GPU** خواهد شد. برای این که بتونیم یک برنامه

طراحی کنیم که از ترکیب **CPU+GPU** پشتیبانی داشته باشد، **NVIDIA** یک مدل برنامه‌نویسی به نام **CUDA** ایجاد کرده است. این مدل برنامه‌نویسی جدید تمرکز کتاب در ادامه خواهد بود.

تفاوت ترد **CPU** با ترد **GPU**

تردها روی **CPU** به عنوان موجودیت‌های سنگینی شناخته می‌شوند. سیستم عامل باید یک ترد را خاموش و روشن کند تا توانایی **multi-threading** را فعال کند. **Context switches** کند و پر هزینه هستند.

تردها بر روی **GPU** خیلی سبک هستند. در یک سیستم عادی، ده ها هزار ترد در صف هستند تا کاری را انجام دهند. اگر یک **GPU** باید منتظر گروهی از تردها باشد، خیلی راحت می‌تواند یک کار دیگر را اجرا کند.

هسته‌های **CPU** طراحی شده‌اند تا میزان **latency** را کمینه کنند، در حالی که هسته‌های **GPU** برای تعداد زیادی ترد موازی طراحی شده‌اند تا قابلیت محاسباتی را بیشینه کنند.

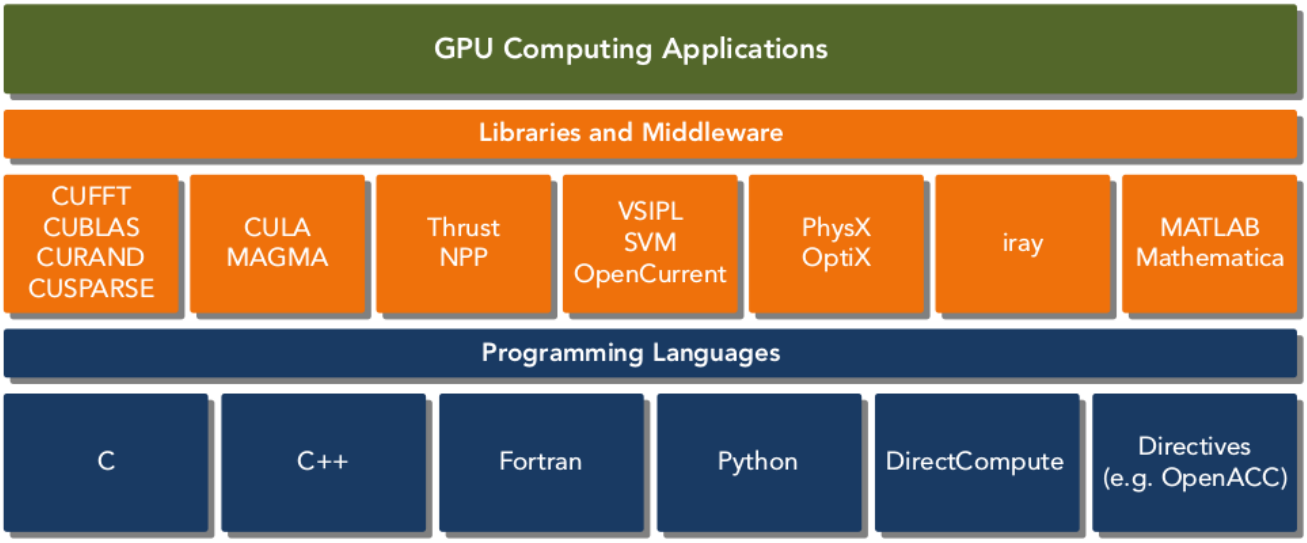
امروزه، یک **CPU** با چهار هسته **quad** می‌تواند تنها ۱۶ تا ترد را به صورت همزمان اجرا کند، البته اگر **CPU** بتواند از **hyper-threading** پشتیبانی کند می‌توان آن را تا ۳۲ تا ترد افزایش داد.

GPU های مدرن **NVIDIA** می‌توانند تا ۱۵۳۶ تا ترد فعال به ازای هر **multiprocessor** داشته باشند. روی یک **GPU** با ۱۶ تا **multiprocessor**، این عدد تا ۲۴۰۰۰ ترد فعال افزایش می‌یابد.

۷ CUDA : یک پلتفرم برای محاسبات ناهمگون

CUDA یک پلتفرم محاسبات موازی **general-purpose** می‌باشد، یک مدل برنامه‌نویسی که موتورهای برنامه‌نویسی موازی در **GPU** های **NVIDIA** را به منظور حل مسایل پیچیده به صورت موثری به کار میگیرد. با استفاده از **CUDA** ، شما می‌توانید به **GPU** به منظور محاسبات دسترسی داشته باشید، که قبلا بر روی **CPU** اجرا میشد.

پلتفرم **CUDA** از طریق کتابخانه‌های شتابدهنده‌ی **CUDA** ، رابط برنامه‌نویسی و افزونه‌هایی که برای زبان‌های استاندارد فراهم شده شامل **C,C++,Fortran** و **python** قابل دسترسی خواهد بود. این کتاب بر روی **CUDA C programming** تمرکز خواهد داشت.

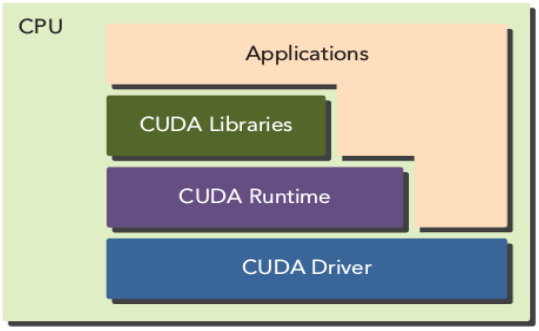


CUDA C یک افزونه از **ANSI C** استاندارد می‌باشد که افزونه‌های زبان به درد بخوری برای فعال کردن برنامه‌نویسی ناهمگون در اختیار ما قرار میدهد، همچنین یک **API** سر راست برای مدیریت **device** ، حافظه، و **task** های دیگر نیز دارد. **CUDA** همچنین یک مدل برنامه‌نویسی **scalable** دارد که می‌شود میزان **scalable** بودن موازی سازی را با تغییر تعداد هسته‌ها تغییر داد، در عین حال نیازی به معلومات پیچیده در مورد **GPU** نخواهد بود و برای کسی که زبان **C** را یاد گرفته این کار آسان خواهد بود.

CUDA دو سطح **API** برای مدیریت **GPU device** و مدیریت تردها فراهم کرده است که بعدا در شکل نیز خواهیم دید:

- CUDA Driver API
- CUDA Runtime API

API مربوط به **driver** یک **API** سطح پایین بوده و تقریبا برای برنامه‌نویسی دشوار می‌باشد، اما کنترل بیشتری بر روی **GPU** در اختیار ما قرار میدهد. **API** مربوط به **runtime** یک **API** سطح بالا بوده که بر روی **API** درایور ساخته شده است. هر تابع از **API** مربوط به **runtime** به عملیات‌های ابتدایی تر که مربوط به **API** درایور است شکسته می‌شود.



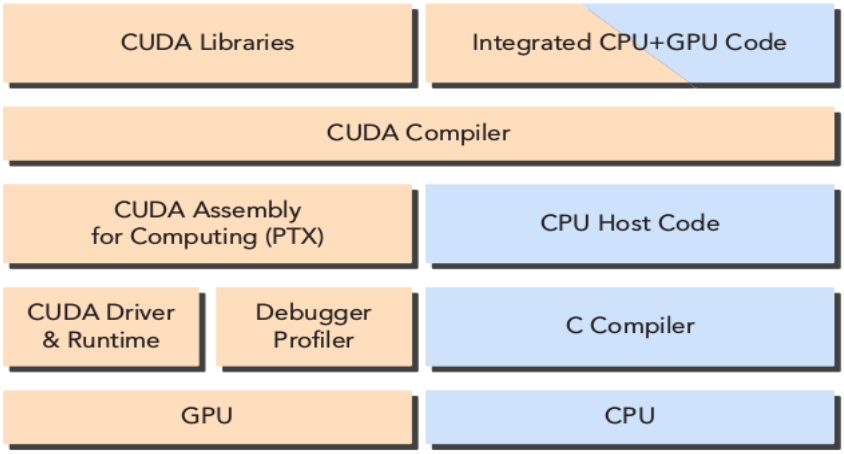
در حقیقت در مورد کارآیی این دو API تفاوت زیادی وجود نـدارد. این کـه چطـور کرنـل شـما از حافظـه استفاده می‌کند و چطور تردها را مدیریت می‌کنید چیزی است که تفاوت کارآیی را مشخص می‌کند.

این دو API دو به دو با همدیگر ناسازگار هستند. شما بایـد یکی از این دو را انتخـاب کنیـد، و این امکـان وجود ندارد که توابع این دو را با هم ترکیب کنید. همه‌ی مثال‌ها در این کتاب از runtime استفاده می‌کنند.

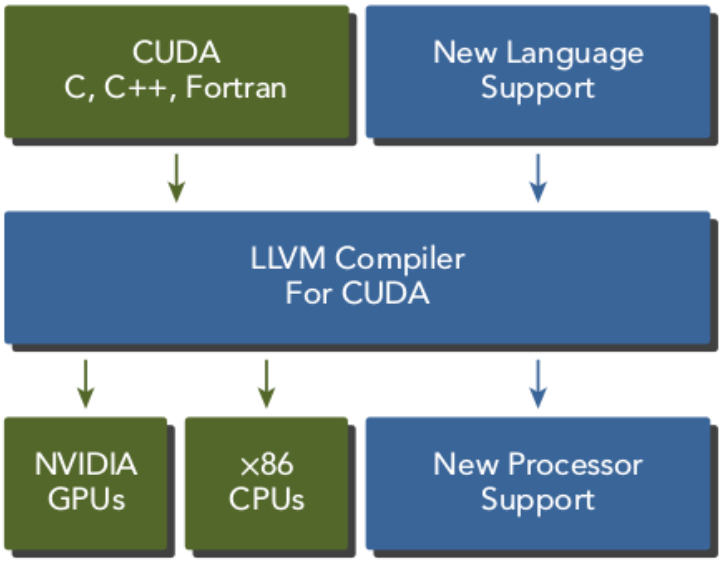
یک برنامه **CUDA** ترکیبی از دو بخش زیر می‌باشد.

- یک کد میزبان که بر روی **CPU** اجرا می‌شود.
- یک کد **device** که بر روی **GPU** اجرا می‌شود.

کامپایلر **nvcc** مربوط به کودای **NVIDIA** در طی مرحله کامپایل کد **device** را از کد میزان جدا می‌کنـد. **Host code** در واقع زبان **C** استاندارد بوده و در ادامه توسط کامپایلر زبان **C** کامپایـل خواهـد شـد. کـد **device** با استفاده از کلمات کلیدی برای مشخص کردن توابـع **data-parallel** جـدا می‌شـود کـه **kernel** نامیده می‌شود. در ادامه کد **device** بر روی **nvcc** کامپایل می‌شـود. در مرحلـه **link**، کتابخـانه‌ی **CUDA runtime** به فراخوان‌های **kernel** اضافه شده و به طور صریح بر روی **GPU** می‌تواند اجرا شود.



کامپایلر **nvcc** عمدتا بر اساس ;کامپایلر **LLVM** نوشته شده است. شـما می‌توانیـد بـا اسـتفاده از **CUDA Compiler SDK** یک زبان برنامه‌نویسی بسازید و یا توسعه دهید که از شتابدهنده‌ی **GPU** پشتیبانی کند.



The CUDA platform is also a foundation that supports a diverse parallel computing ecosystem, as shown in Figure 1-16. Today, the CUDA ecosystem is growing rapidly as more and more companies provide world-class tools, services, and solutions. If you want to build your applications on GPUs,the easiest way to harness the performance of GPUs is with the CUDA Toolkit (https://developer.nvidia.com/cuda-toolkit), which provides a comprehensive development environment for C and C++ developers. The CUDA Toolkit includes a compiler, math libraries, and tools for debugging and optimizing the performance of your applications. You will also fi nd code samples,programming guides, user manuals, API references, and other documentation to help you get started.

اولین کدمان را به صورت زیر می‌نویسیم.

```
#include <stdio.h>
#include <iostream>
using namespace std;
__global__ void hellofromGPU(void)
{
    printf("Hello world from GPU!! \n");
}
int main()
{
    cout<<"Hello world from CPU!!"<<endl;
    hellofromGPU<<<1,1>>>();
    cudaDeviceReset();
    return 0;
}
```

در این کد ما تابع **hellofromGPU** را به صورت زیر تعریف کرده‌ایم.

```
__global__ void hellofromGPU(void)
{
   printf("Hello world from GPU!! \n");
}
```

این تابع یک تابع کرنل است که در آن **__global__** یک **qualifier** به حساب می‌آید. این **qualifier** به کامپایلر می‌گوید که این تابع از روی **CPU** خوانده می‌شود و روی **GPU** اجرا می‌شود. برای اجرا کردن تابع کرنل باید به صورت زیر عمل کنیم.

```
hellofromGPU<<<1,10>>>();
```

در این کد **triple angle bracket** یک فراخوانی از ترد میزبان به کدی که بر روی **device** قرار دارد مارک می‌کند. یک کرنل توسط آرایه‌ای از تردها اجرا شده و همه‌ی تردها یک کد یکسان را اجرا خواهند کرد. پارامترهایی که درون براکت سه‌گانه آورده شده است، پارامترهای تنظیمی برای اجرا می‌باشد، که مشخص کننده‌ی تعداد تردهایی است که تابع کرنل را اجرا می‌کنند. در این مثال، شما از ۱۰ ترد برای اجرای کرنل استفاده می‌کنید. و خروجی به صورت زیر خواهد بود.

```
Hello world from CPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
```

تابع **cudaDeviceReset()** صریحا همه‌ی منابعی که این پردازش از **device** گرفته را تخریب و پاک می‌کند. برای کامپایل بر روی معماری **sm_20** به صورت زیر نیز می‌توانیم عمل کنیم.

```
nvcc -arch sm_20 hello.cu -o hello
```

یک برنامه‌ی عادی کودا شامل پنج قدم خواهد بود.

۱. اختصاص دادن حافظه‌ی **GPU**

۲. کپی کردن داده از روی حافظه‌ی **CPU** روی حافظه‌ی **GPU**.

۳. فراخوانی کردن یک تابع کرنل کودا برای انجام یک محاسبه‌ی خاص.

۴. کپی کردن داده از حافظه‌ی **GPU** روی حافظه‌ی **CPU**.

۵. نابود کردن حافظه‌ای که به **GPU** اختصاص داده بودیم.

۸ آیا برنامه‌نویسی با CUDA مشکل است؟

تفاوت عمده بین برنامه‌نویسی **CPU** و برنامه‌نویسی **GPU** در میزان آشنایی است که با ویژگی‌های معماری **GPU** باید کسب کرد. فکر کردن به صورت موازی و داشتن یک درک اولیه از معماری **GPU** به شما این امکان را میدهد که برنامه‌های موازی خودتان را بنویسید .

اگر می‌خواهید که به عنوان یک مهندس برنامه‌های موازی یک کد موثر و با سرعت بالا بنویسید، نیاز به یک آشنایی اولیه با معماری **CPU** نیز خواهید داشت. به طور مثال، **locality** یک مفهوم خیلی مهم در برنامه‌نویسی موازی است. در واقع **Locality** اشاره به استفاده مجدد از داده‌ها برای کاهش تاخیر دسترسی به حافظه می‌باشد. **Temporal Locality** یعنی این که داده و یا منابع به مدت زمان کوتاهی قابل استفاده مجدد باشد و **Spatial Locality** یعنی این که داده‌های یک المان در حافظه نزدیک به همدیگر باشند. **CPU** های مدرن از کش‌های بزرگ استفاده می‌کنند، تا برنامه‌هایی که **Spatial** و **Temporal locality** مناسبی دارند در بهترین حالت خود اجرا شوند. در واقع این مسوولیت برنامه‌نویس است که برنامه را طوری بنویسید که به بهترین نحو از **CPU cache** استفاده بکند. برنامه‌نویسان باید بهینه‌سازی‌های سطح پایین **cache** را در دست بگیرند، ولی نیازی به این نیست که در مورد تردهای **CPU** و چگونگی اجرای این تردها فکر کنند چون که **CPU** همچین اطلاعاتی را به ما نمیدهد.

CUDA در این مورد هم سلسله مراتب حافظه و هم سلسه مراتب تردها را در اختیار ما قرار میدهد، و توانایی کنترل اجرای تردها و برنامه‌ریزی تردها را به درجه‌ی بالاتری می‌رساند.

به طور مثال، ما یک حافظه‌ی خاص داریم که **shared memory** نامیده می‌شود، که توسط برنامه‌نویسی **CUDA** قابل کنترل است. **Shared memory** را می‌توان به عنوان کش قابل کنترل توسط برنامه دید، که افزایش سرعت قابل ملاحظه‌ای را با حفظ کردن پهنای باند **main memory** به ما میدهد. با استفاده از حافظه **share** شده، می‌توانید **locality** کدتان را مستقیما کنترل کنید.

وقتی که یک برنامه توسط **ANSI C** نوشته می‌شود، شما باید صریحا تردهای خودتان را با **pthreads** و یا **OpenMP** کنترل کنید، این دو تکنیک‌های شناخته شده‌ای هستند که برنامه‌نویسی موازی را در اکثر معماری‌ها و سیستم عامل‌ها پشتیبانی می‌کند. وقتی که یک برنامه در **CUDA C** نوشته می‌شود، در واقع یک تکه کد سریال نوشته‌اید که توسط یک ترد اجرا می‌شود. **GPU** این کرنل را گرفته و آن را با اجرا کردن بر روی هزاران ترد تبدیل به یک کد موازی می‌کند، در واقع همه‌ی تردها یک کد را اجرا می‌کنند.

مدل برنامه‌نویسی موازی **CUDA** راهی برای کنترل تردهـای معمـاری میدهـد. دسـتکاری مسـتقیم این سازماندهی روی ترتیب تردهایی که در **GPU** اجرا می‌شود تاثیر میگذارد. از آنجایی کـه **C CUDA** یـک افزونه از زبان **C** است، معمولا تبدیل کد زبان **C** به **CUDA C** کار سختی نخواهد بود.

CUDA جزییات سخت‌افزاری را خود حل می‌کند و نیازی نیست که برنامه به **API** سابق نگاشت شود. در هسته‌ی **CUDA** ما سه نوع مفهوم ابسترکت شده داریم: سلسله مراتب گروه‌های ترد، یک سلسله مـراتب از گروه‌های حافظه، و یک حایل سنکرون سازی، که به صورت حداقل چیزی که از افزونه‌هـا نیـاز داریـد ارایه شده است. با هر نسخه‌ی **CUDA**، برنامه‌نویسی موازی توسط **NVIDIA** سـاده‌تر شـده. البتـه بـرخی هنوز مفاهیم **CUDA** را سطح پایین می‌دانند، و بالا بردن این ابسترکت ممکن است که توانـایی شـما در کنترل برنامه تان را کاهش دهد. بدون این کنترل، کارآیی برنامه شما دیگر در کنترل شما نخواهـد بـود. بنابراین، چالشی که باید با آن روبه‌رو شوید آشنایی اولیه بـا معمـاری **GPU** و خـبره شـدن در ابزارهـای توسعه **CUDA** و محیط توسعه می‌باشد.

۹ محیط توسعه‌ی CUDA

NVIDIA محیط توسعه‌ی بزرگی را برای توسعه برنامه‌هایی که نیاز به **GPU** دارند، توسط **C** و **C++** آماده کرده است که شامل:

- NVIDIA NsightTM integrated development environment
- CUDA-GDB command line debugger
- Visual and command line profiler for performance analysis
- CUDA-MEMCHECK memory analyzer
- GPU device management tools

پس از آشنایی با این ابزارها، برنامه‌نویسی با زبان **C** یک کار آسان خواهد بود.

***Locality* in this sense is speaking mostly to *cache locality*. Writing data structures and algorithms to operate mostly out of cache makes the algorithm run as fast as it possibly can. Cache locality is one of the reasons *quick sort* is quick.**

For a data structure, you want to keep the parts of your data structure that refer to each other relatively close to each other, to avoid flushing out useful cache lines.

Also, you can rearrange your data structure so that the compiler will use the minimum amount of memory required to hold all the members and still efficiently access them. This helps make sure your data structure consumes the minimum number of cache lines.

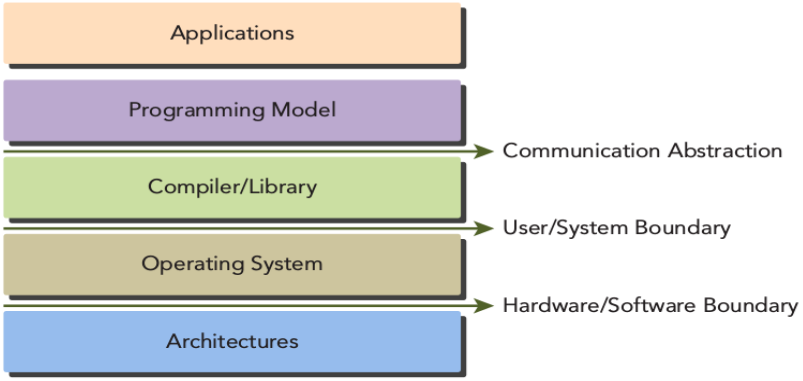
A single cache line on a current x86-64 architecture (core i7) is 64 bytes.

فصل دوم : مدل برنامه‌نویسی CUDA

CUDA یک پلتفرم محاسبات موازی و یک مدل برنامه‌نویسی است که همراه با مجموعه‌ی کوچکی از افزونه‌ها که به زبان **C** افزوده شده. با استفاده از **CUDA**، می‌توانید به سادگیه نوشتن یک برنامه‌ی زبان **C**، کد نویسی کنید. با استفاده از **CUDA** می‌توانید برای هزاران سیستم که از کارت گرافیک **NVIDIA** استفاده می‌کنند برنامه بنویسید، که محدوده‌ای از دیوایس‌های نهفته، تبلت‌ها، لپ‌تاپ‌ها، سیستم‌های دسکتاپ تا **workstation** برای سیستم‌های کلاستر **HPC** را شامل می‌شود. ابزارهای برنامه‌نویسی مشابه زبان **C** توسعه داده شده اند تا در امر تصحیح، دیباگ و آنالیز برنامه به شما کمک کنند. در این فصل، یاد خواهید گرفت تا چطور یک برنامه **CUDA** بنویسید، این کار را با دو مثال ساده توضیح خواهیم داد: جمع برداری و جمع ماتریسی این دو برنامه ما خواهند بود.

۱ معرفی مدل برنامه‌نویسی CUDA

مدل برنامه‌نویسی یک **abstraction** از معماری کامپیوتر را بیان می‌کند که به عنوان یک پل فی ما بین برنامه و پیاده‌سازی روی سخت افزار عمل می‌کند. شکل زیر لایه‌های مهم این **abstraction** را که بین برنامه و پیاده‌سازی مدل برنامه‌نویسی خوابیده است را نشان می‌دهد.



Abstraction ارتباط، یک مرز بین برنامه و پیاده‌سازی مدل برنامه‌نویسی است، که از طریق کامپایلر یا کتابخانه توسط سخت افزار و سیستم عامل فهمیده می‌شود. برنامه ای که توسط مدل برنامه‌نویسی نوشته می‌شود، نشان‌دهنده‌ی کامپوننت‌هایی است که برنامه به اشتراک می‌گذارد. مدل برنامه‌نویسی یک دید منطقی از معماری محاسباتی مشخصی را ارایه میدهد. معمولا، آن را در زبان برنامه‌نویسی و یا محیط برنامه‌نویسی قرار داده است.

علاوه بر این **abstraction** هایی که با سایر مدل‌های برنامه‌نویسی موازی بـه اشـتراک گذاشـته می‌شـود، مدل برنامه‌نویسی **CUDA** ویژگی‌های خاص زیر را برای دسترسی به قدرت بیشتر محاسـباتی در اختیـار قرار میدهد.

- راهی برای مدیریت تردها بر روی **GPU** از طریق ساختار سلسله مراتبی.
- راهی برای دسترسی به حافظه بر روی **GPU** از طریق ساختار سلسله مراتبی.

ما در اولین بحثی که در این فصل و فصل بعد خواهیم داشت روی مورد اول تمرکز خواهیم کرد، و مـورد دوم را در فصل چهار و پنج مورد بررسی قرار خواهیم داد.

از دید یک برنامه‌نویس، می‌توانید برنامه‌نویسی موازی را در سطوح مختلفی ببینید، مثل:

- سطح **Domain**
- سطح منطقی
- سطح سخت افزاری

وقتی که دارید برنامه‌نویسی انجام میدهید و الگوریتم را طراحی میکنیـد، توجه‌تـان معطـوف بـه **سطح domain** می‌باشد: چطور داده‌ها را تقسیم کنیم و چطور توابع رو برای حل مساله مان در حالت موازی بـه صورت کارآمدی به کار بگیریم، این **سطح domain** است. وقتی که وارد فاز برنامه نویسی شدید، باید بـه این فکر کنید که چطور تردهای موازی را مدیریت کنیـد، در طی این فـاز شـما در **سطح منطقی** فکـر می‌کنید تا اطمینـان حاصـل کنیـد کـه تردهـا و محاسـبات مسـاله را درسـت حـل می‌کننـد. در زبـان برنامه‌نویسی موازی که توسط **C** عادی انجام می‌شود، شما باید به طور صریح از **pthreads** و یا **OpenMP** استفاده کنید. **CUDA** این **abstraction** ای که در مورد کنترل رفتار تردها وجود دارد را در اختیار ما قرار داده. وقتی که مثال‌های این کتاب رو می‌بینیم، خواهید دید که چنین **abstraction** ای چه مقیاس‌پذیری فوق‌العاده‌ای را به ما می‌دهد. در **سطح سخت افزار**، قادر به درک چگونگی نگاشت شدن تردها بـر روی هسته‌ها خواهیم بود، شاید این موضوع به بهبود دادن کارآمدی سیستم کمک کند.

مدل ترد **CUDA** اطلاعات کافی را در اختیار شما قرار میدهد بدون این کـه شـما را مجبـور بـه دانسـتن جزییات سطح پایین کند. در این مورد در فصل ۳ بیشتر خواهیم دانست.

۲ ساختار برنامه‌نویسی CUDA

مدل برنامه‌نویسی **CUDA** به شما این امکان را میدهد که برنامه‌تان را بر روی سیستم‌های ناهمگون اجرا کنید، این کار با نشان‌دار کردن کد با یک مجموعه‌ی کوچک از افزونه‌هایی که بـه زبـان **C** افـزوده شـده انجام میپذیرد. یک محیط ناهمگون متشکل شده از **CPU** ها به همراه **GPU** می‌باشد، که هر کـدام دارای

حافظه‌ی اختصاصی است که با گذرگاه **PCI-Express** از همدیگر جدا شده‌اند. بنابراین، باید متوجه تفاوتی که این دو حافظه دارند باشید:

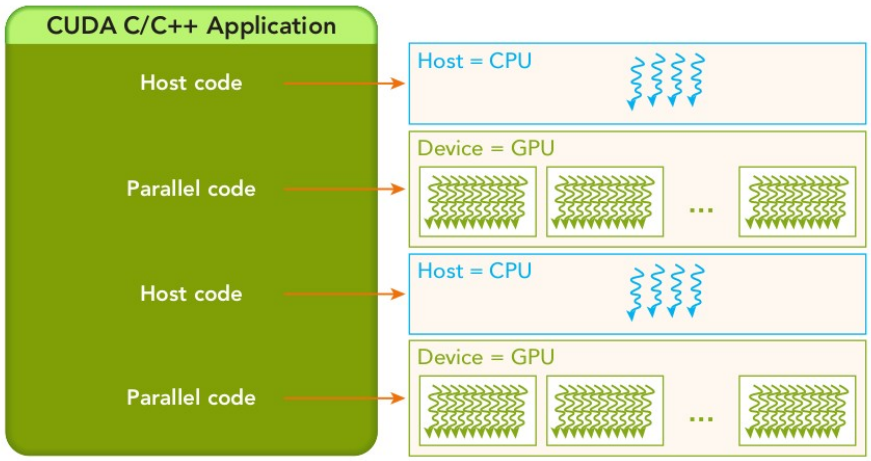
- **Host : CPU** و حافظه اختصاصی خودش(حافظه‌ی **host**)
- **Device : GPU** و حافظه‌ی اختصاصی(حافظه‌ی **device**)

برای مشخص شدن تفاوت این دو حافظه، متغیرهایی که با **_h** شروع می‌شوند برای حافظه‌ی **host** و متغیرهایی که با **_d** شروع می‌شوند برای حافظه‌ی **device** به کار می‌رود.

با شروع **CUDA 6**، **NVIDIA** یک بهبود در مدل برنامه‌نویسی داد که **Unified Memory** (حافظه‌ی یکپارچه) نامیده می‌شود، که به عنوان یک پل بین حافظه‌ی **host** و **device** کار می‌کند. این توسعه به شما این اجازه را میدهد که هم به حافظه‌ی **CPU** و هم **GPU** با یک اشاره‌گر دسترسی داشته باشید، در حالی که سیستم به صورت اوتوماتیک داده‌ها را بین **host** و **device** جابه‌جا می‌کند. اطلاعات بیشتر در مورد این حافظه در فصل چهارم توضیح داده خواهد شد. قبل از آن شما نیاز دارید که یاد بگیرید چطور روی **host** و **device** حافظه اختصاص دهید، و صریحا داده را بین **CPU** و **GPU** جابه‌جا کنید. این کنترلی که برنامه‌نویس بر روی حافظه و داده دارد می‌تواند باعث بالابردن حداکثری کارآمدی سیستم و استفاده بیشینه از سخت افزار شود.

یک کامپوننت کلیدی در مدل برنامه‌نویسی **CUDA** کرنل می‌باشد(کرنل کدی است که بر روی **GPU** اجرا می‌شود. به عنوان یک توسعه دهنده، شما می‌توانید یک کرنل را به صورت یک برنامه‌ی ترتیبی بنویسید. در پشت صحنه، **CUDA** این کرنل نوشته شده توسط برنامه‌نویس را برای اجرا بر روی تردهای **GPU** مدیریت می‌کند. از طریق **host** ، تعریف می‌کنید که چطور الگوریتم بر روی **device** بر اساس داده‌ها نگاشت شود. در این صورت ما درگیر مدیریت هزاران ترد **GPU** نخواهیم شد.

Host می‌تواند برای بیشتر عملیات‌ها به طور مستقل از **device** کار کند. وقتی که یک **kernel** اجرا می‌شود، کنترل بلافاصله دست **host** میفتد، و **CPU** را آزاد نگه می‌دارد تا محاسبات موازی بر روی **device** اجرا شود. مدل برنامه‌نویسی **CUDA** یک پروسه آسنکرون می‌باشد، بنابراین محاسبات **GPU** بر روی **GPU** می‌تواند با ارتباطات **host** همپوشانی پیدا می‌کند. یک برنامه‌ی عادی **CUDA** متشکل از یک کد سریال هست که با یک کد موازی تکمیل شده است. همانطور که در شکل نمایش داده شده است، کد ترتیبی بر روی **host** اجرا می‌شود، ولی کد موازی بر روی دیوایس **GPU** اجرا می‌شود. کد **host** با استفاده از زبان **C** نوشته شده در حالی که کد **device** با استفاده از زبان **CUDA C** نوشته شده است. می‌توانید همه‌ی کد را در داخل یک سورس فایل بنویسید، یا این که می‌توانید چندین سورس فایل داشته باشد. کامپایلر **NVIDIA(NVCC)** هم برای **host** و هم برای **device** کد را تولید می‌کند.



یک روش معمول برای پردازش برنامه **CUDA** روش زیر می‌باشد.

۱. کپی کردن داده از **CPU** بر روی حافظه‌ی **GPU**

۲. فراخوانی کردن کرنل برای پردازش داده‌هایی که در **GPU** ذخیره شده است.

۳. کپی کردن حافظه از **GPU** به **CPU** .

ما کار را با مدیریت حافظه و تبادل اطلاعات بین **host** و **device** آغاز خواهیم کرد. در ادامه، این فصل جزییات بیشتری در مورد کرنل‌های **GPU** خواهیم دید.

۳ مدیریت حافظه

مدل برنامه‌نویسی **CUDA** فرض می‌کند که سیستم از یک **host** و از یک **device** تشکیل شده است، که هر کدام دارای حافظه‌ی جداگانه خود هستند. کرنل در خارج از حافظه‌ی **device** اجرا می‌شود تا این امکان را به شما بدهد که کنترل کاملی داشته باشید و به بهترین کارآمدی برسید، **CUDA runtime** توابعی رو برای اختصاص حافظه‌ی **device**، رها کردن حافظه‌ی **device** ، و انتقال اطلاعات بین حافظه‌ی **host** و **device** را فراهم کرده است. جدول زیر یک لیست از توابع استاندارد **C** و تابع منطبق در **CUDA C** را فراهم کرده است.

TABLE 2-1: Host and Device Memory Functions

STANDARD C FUNCTIONS	CUDA C FUNCTIONS
malloc	cudaMalloc
memcpy	cudaMemcpy
memset	cudaMemset
free	cudaFree

تابعی که برای اختصاص حافظه در **GPU** استفاده می‌شود **cudaMalloc** می‌باشد، و نوع استفاده از تابع به صورت زیر می‌باشد.

```
cudaError_t cudaMalloc ( void** devPtr, size_t size );
```

این تابع یک رنج خطی از حافظه **device** به میزانی که مشخص شده میگیرد. حافظه گرفته شده به بایت می‌باشد. حافظه اختصاص داده شده توسط **devPtr** قابل بازیابی خواهد بود. احتمالا متوجه تشابهی که بین **cuda-malloc** و **malloc** در زبان **C** است پی برده‌اید. این تشابه، عمدی بوده. با نزدیک نگه داشتن رابط به زبان برنامه‌نویسی **C** ، استفاده از **CUDA** آسان تر خواهد بود.

تابعی که برای انتقال داده بین **host** و **device** مورد استفاده قرار میگیرد **cudaMemcpy** می‌باشد، و نحوه‌ی استفاده از این تابع نیز به صورت زیر می‌باشد.

```
cudaError_t cudaMemcpy(void* dst,const void* src,size_t count,cudaMemcpyKind kind);
```

این تابع به میزان بایت‌های مشخص شده از قسمت حافظه‌ی سورس، که توسط **src** به آن اشاره شده، به فضای حافظه‌ی مقصد که با **dst** به آن اشاره شده یک کپی انجام میدهد. توجه شود که نوع **cudaMemcpyKind** به چهار دسته زیر تقسیم می‌شود:

- cudaMemcpyHostToHost
- cudaMemcpyHostToDevice
- cudaMemcpyDeviceToHost
- cudaMemcpyDeviceToDevice

این تابع یک رفتار آسنکرون نخواهد داشت، چون برنامه‌ی **host** تا وقتی که تابع کارش تمام نشود بلاک می‌ماند. هر فراخوانی از **CUDA**، به جز فراخوانی کرنل، یک کد خطا به صورت **enumerated** شده در **cudaError_t** برمیگردلند. به طور مثال، اگر حافظه‌ی **GPU** به درستی تخصیص داده شده باشد، چیزی که این تابع برمیگرداند به صورت زیر خواهد بود.

cudaSuccess

در غیر این صورت چیزی که برمیگرداند به صورت زیر هست.

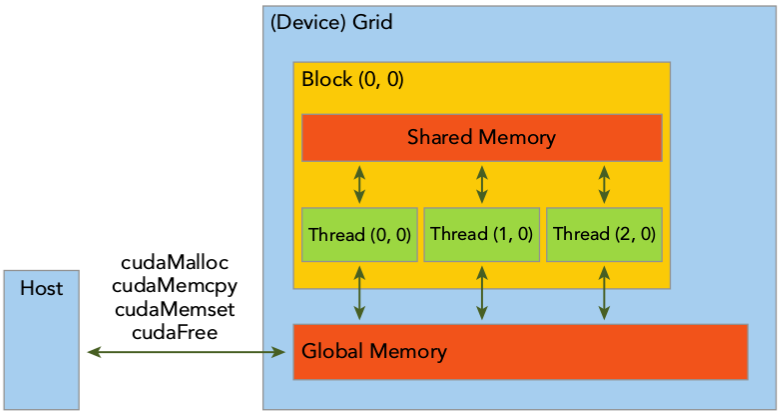
cudaErrorMemoryAllocation

برای تبدیل این کد خطا به چیزی که برای انسان قابل خواندن باشد می‌توانید از این تابع **run-time** استفاده کنید.

```
const char* string=cudaGetErrorString(cudaError_t  Error);
```

تابع **cudaGetErrorString** مشابه تابع **strerror** در زبان **C** استاندارد می‌باشد.

مـدل برنامه‌نویسـی **CODA** یـک **abstraction** از سلسـله مـراتب حافظـه را از معمـاری **GPU** در اختیـار میگذارد. شکل زیر یک ساختار ساده شده حافظه‌ی **GPU** را نشان میدهد، که شامل دو زیر شاخه‌ی مهم است: حافظه‌ی **global** و حافظه‌ی **share** شده. در مورد ساختار سلسله مراتبی **GPU** در فصل چهار و پنج بیشتر یاد خواهیم گرفت.

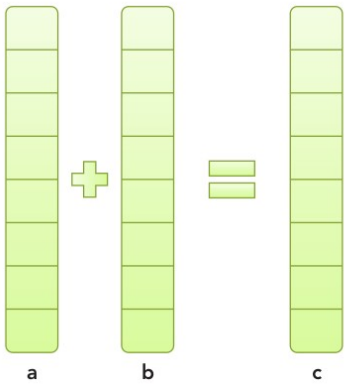


۴ سلسله مراتب حافظه GPU (خلاصه)

یکی از ویژگی‌های خیلی خاص برنامه‌نویسی **CODA** اطلاعاتی است که در مورد سلسله مراتب حافظه در اختیار ما قرار داده شده. هر **GPU** دارای مجموعه حافظه‌ی متفاوتی می‌باشد که برای کاربردهای متفاوت انتخاب شده است. در این مورد جزییات این بحث در فصل چهار و پنج بیشتر بحث خواهیم کرد.

در سلسله مراتب حافظه‌ی **GPU**، دو حافظه‌ی مهم وجود دارد که بـه عنـوان حـافظه‌ی **global** و **shared** شناخته می‌شود. حافظه‌ی **global** مشـابه حـافظه‌ی **CPU** بـوده، در حـالی کـه **shared memory** شـبیه حافظه‌ی کش **CPU** می‌باشد. ولی با این تفاوت که، حافظه‌ی **shared** را می‌توان به طور مستقیم با کرنـل کنترل کرد.

فعلا قرار هست که به مدیریت داده بین **host** و **device** بپردازیم، و یاد بگیریم که چطور از طریق **CODA C** دو آرایه را با همدیگر جمع کنیم. همانطور که در شکل بعـدی مشـخص اسـت، المـان اول از آرایه‌ی **a** با المان اول از آرایه‌ی **b** جمع می‌شود، و نتیجه در المان اول در آرایه‌ی **c** گذاشته می‌شود. این محاسبات برای همه‌ی المان‌ها تکرار می‌شود.



برای جمع زدن دو آرایه ابتدا با **host code** شروع می‌کنیم(**C** استاندارد).

```
#include <stdlib.h>
#include <string.h>
#include <time.h>
void sumArrayOnHost(float *A,float *B,float *C,const int N)
{
    for (int idx = 0; idx <N; ++idx)
    {
        C[idx]=A[idx] + B[idx];
    }
}
void initialData(float *ip,int size)
{
    //generate different seed for random number
    time_t t;
    srand((unsigned int) time(&t));
    for (int i = 0; i < size; ++i)
    {
        ip[i] = (float) (rand() & 0xFF) /10.0f;
    }
}
int main()
{
    int nElem=1024;
    size_t nBytes = nElem * sizeof (float);
    float *h_A,*h_B,*h_C;
    h_A = (float *)malloc(nBytes);
    h_B = (float *)malloc(nBytes);
```

```
    h_C = (float *)malloc(nBytes);
    initialData(h_A, nElem);
    initialData(h_B, nElem);
    sumArrayOnHost(h_A, h_B, h_C,nElem);
    free(h_A);
    free(h_B);
    free(h_C);
    return(0);
}
```

از آنجایی که این یک برنامه خالص زبان **C** هست، می‌تونید با کامپایلر زبان **C** کامپایلش کنید. همچنین می‌تونید با استفاده از **nvcc** به صورت زیر کامپایلش کنید.

```
$ nvcc -Xcompiler -std=c99 sumArraysOnHost.c –o sum
$ ./sum
```

کامپایلر **CODA** به شما این امکان را میدهد که آپشن‌های فراوانی را به طور مستقیم به **nvcc** پاس دهید. فلگ **Xcompiler**– آپشن‌ها را مستقیما برای کامپایلر **C** مشخص می‌کند. در این مثال، **std=c99**– به کامپایلر پاس داده شده است چون کد نوشته شده طبق استاندارد **C99** نوشته شده است. شما می‌توانید این آپشن‌ها را در داکیومنت کامپایلر **CODA** در آدرس زیر پیدا کنید.

```
http://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/index.html
```

حال می‌توانیم کد را به نحوی تغییر دهیم تا بر روی **GPU** اجرا شود.

```
    float *d_A, *d_B, *d_C;
    cudaMalloc((float**)&d_A, nBytes);
    cudaMalloc((float**)&d_B, nBytes);
    cudaMalloc((float**)&d_C, nBytes);
```

حال از تابع **cudaMemcpy** برای انتقال اطلاعات از حافظه‌ی **host** به حافظه‌ی **global** کارت گرافیک استفاده می‌کنیم، پارامتر **cudaMemcpyHostToDevice** مشخص کننده ی نوع انتقالی است که قرار است صورت بگیرد.

```
    cudaMemcpy(d_A,h_A,nElem,cudaMemcpyHostToDevice);
    cudaMemcpy(d_B,h_B,nElem,cudaMemcpyHostToDevice);
```

وقتی که داده به حافظه‌ی **global** کارت گرافیک انتقال داده شد، تابع کرنل می‌تواند از طرف **host** برای جمع دو آرایه بر روی **GPU** صدا زده شود. به محض این که کرنل صدا زده شود، سریعا کنترل به دست **host** برمیگردد. در این نقطه، **host** می‌تواند توابع دیگری را اجرا کند و در همین زمان **GPU** نیز پردازش خودش را انجام بدهد. بنابراین **kernel** نسبت به **host** یک پردازش آسنکرون انجام می‌دهد.

وقتی که کرنل کار پردازش بر روی همه‌ی المان‌ها را تمام کرد، نتیجه بر روی حافظه‌ی گلوبال **GPU** و در آرایه‌ی **d_c** ذخیره شده است. حال باید نتیجه‌ی عملیات **GPU** را به آرایه **gpuRef** بر روی **host** برگردانیم و این کار دوباره با استفاده از تابع **cudaMemcpy** انجام میپذیرد.

```
float *gpuRef;
cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyDeviceToHost);
```

فراخوانی کردن **cudaMemcpy** موجب بلاک شدن پردازش **host** می‌شود. نتیجه‌ای که بر روی آرایه‌ی **d_c** بر روی **GPU** قرار داد به **gpuRef** کپی می‌شود، آرگومان **cudaMemcpyDeviceToHost** به این منظور استفاده شده. در نهایت، حافظه‌ای که بر روی **GPU** گرفته شده بود را با استفاده از تابع **cudaFree** آزاد می‌کنیم.

```
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
```

۴ تفاوت فضای حافظه host و device

یکی از رایج‌ترین اشتباهاتی که یک برنامه‌نویس تازه کار در **CODA C** انجام میدهد که به صورت اشتباهی بین حافظه‌های **dereference** کند. برای حافظه‌ای که بر روی **GPU** اختصاص داده شده، پوینتر ممکن است به درستی بر روی هاست **dereference** شود. اگر کد شما به طرز اشتباهی عملیات انتساب زیر را انجام بدهد:

```
gpuRef=d_C;
```

به جای این که از شکل درست زیر استفاده کنید:

```
cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyDeviceToHost);
```

برنامه شما ممکن است در **runtime** با خطا روبه‌رو شده و کرش کند. برای کمک به این مشکل در **CODA 6** یک حافظه‌ی یکپارچه معرفی شده که به شما اجازه می‌دهد که هم بر روی **CPU** و هم بر روی **GPU** با استفاده از یک اشاره‌گر تنها دسترسی داشته باشید. در مورد این حافظه یکپارچه در فصل چهارم بیشتر یاد خواهیم گرفت.

۵ مدیریت تردها

```
To help avoid these types of mistakes, Unifi ed Memory was introduced with CUDA
6, which lets you access both CPU and GPU memory by using a single pointer. You
will learn more about unifi ed memory in Chapter 4.
```